

DBMS – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Skill Task: DBMS

Week: 6

Objectives

1. Create the Review table with a rating column.
2. Store customer feedback and ratings.
3. Retrieve product review details.
4. Calculate average product ratings using aggregate functions.
5. Identify highly rated products.

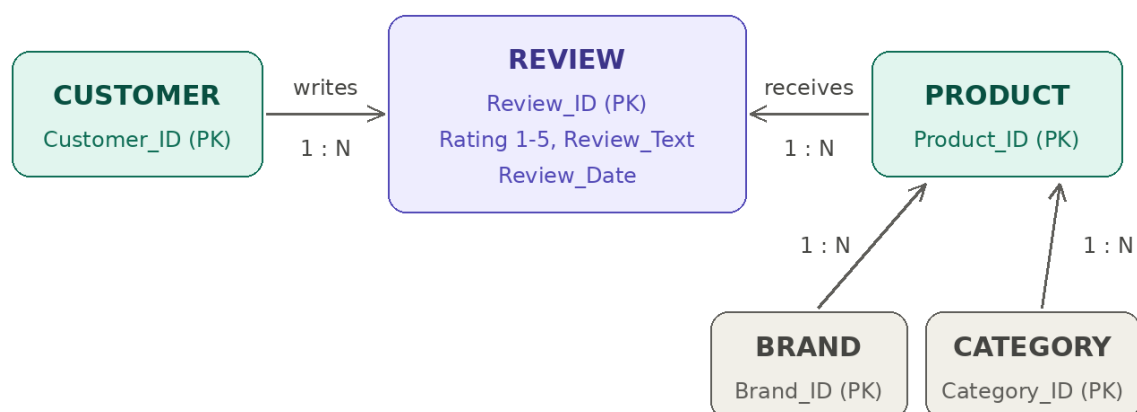
1. Database Design

1.1 Review Table

Attribute	Data Type	Constraint
Review_ID	NUMBER(5)	Primary Key
Customer_ID	NUMBER(5)	Foreign Key (Customer)
Product_ID	NUMBER(5)	Foreign Key (Product)
Rating	NUMBER(1)	NOT NULL, CHECK (1–5)
Review_Text	VARCHAR2(500)	NOT NULL
Review_Date	DATE	NOT NULL

The rating (1 to 5) is stored in the Rating column of the Review table, as defined in the entity analysis.

2. Table Relationships



Customer and Product each have a one-to-many relationship with Review. Brand and Category each have a one-to-many relationship with Product.

3. Table Creation

```
SQL> CREATE TABLE Review ( Review_ID NUMBER(5) PRIMARY KEY, Customer_ID NUMBER(5), Product_ID
NUMBER(5), Rating NUMBER(1) NOT NULL, Review_Text VARCHAR2(500) NOT NULL, Review_Date DATE NOT
NULL, CONSTRAINT chk_review_rating CHECK (Rating BETWEEN 1 AND 5), CONSTRAINT
fk_review_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID), CONSTRAINT
fk_review_product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID) );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (601, 101, 101, 5, 'Perfect fit and great fabric quality', SYSDATE-5);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (602, 102, 102, 4, 'Stylish design and good stitching', SYSDATE-4);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (603, 103, 101, 5, 'Very comfortable and true to size', SYSDATE-3);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (604, 104, 103, 3, 'Decent product, colour slightly faded', SYSDATE-2);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (605, 101, 103, 4, 'Good value for the price', SYSDATE-1);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Rating, Review_Text, Review_Date)
VALUES (606, 102, 101, 4, 'Loved it, delivered on time', SYSDATE);
```

1 row created.

```
SQL> COMMIT;
```

Commit complete.

5. Review and Rating Data Verification

```
SQL> SELECT Review_ID, Customer_ID, Product_ID, Rating, Review_Date FROM Review ORDER BY Review_ID;
```

6. Retrieve Product Review Details

```
SQL> SELECT r.Review_ID, c.First_Name || ' ' || c.Last_Name AS Customer_Name, p.Product_Name,
r.Rating, r.Review_Text, r.Review_Date FROM Review r JOIN Customer c ON r.Customer_ID =
c.Customer_ID JOIN Product p ON r.Product_ID = p.Product_ID ORDER BY r.Review_Date DESC;
```

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```
SQL> SELECT Product_ID, ROUND(AVG(Rating), 2) AS Average_Rating FROM Review GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
101	4.67
102	4
103	3.5

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SQL> SELECT Product_ID, ROUND(AVG(Rating), 2) AS Average_Rating FROM Review GROUP BY Product_ID
HAVING AVG(Rating) >= 4 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	AVERAGE_RATING
101	4.67
102	4

9. Product Rating Report

A combined report can be generated using the Review, Product and Brand tables.

```
SQL> SELECT p.Product_ID, p.Product_Name, b.Brand_Name, COUNT(r.Review_ID) AS Total_Reviews,
ROUND(AVG(r.Rating), 2) AS Average_Rating, MAX(r.Rating) AS Highest_Rating, MIN(r.Rating) AS
Lowest_Rating FROM Review r JOIN Product p ON r.Product_ID = p.Product_ID JOIN Brand b ON
p.Brand_ID = b.Brand_ID GROUP BY p.Product_ID, p.Product_Name, b.Brand_Name ORDER BY
Average_Rating DESC;
```